
Undeniable Signature Schemes on Modular Lattices

A PROJECT REPORT

Submitted By

Deepesh Garg
Jyoti Chauhan
Mayank Kumar

20114032
20114042
20114055

*For the fulfillment
of*
CSN-400A: B.Tech. Project

Under the Guidance of
Professor Sugata Gangopadhyay



Department of Computer Science and Engineering
INDIAN INSTITUTE OF TECHNOLOGY ROORKEE
ROORKEE-247667
MAY 2024

Acknowledgment

I would like to sincerely thank Prof Sugata Gangopadhyay and Mr. Ali Raya for their valuable guidance and support in completing our project and for giving us this great opportunity to do a project on Undeniable signature schemes on modular lattices. Without their support and suggestions, this project would not have been completed.

Department of Computer Science and Technology

Date: 07/05/2024

Deepesh Garg, 20114032

Jyoti Chauhan, 20114042

Mayank Kumar, 20114055

Contents

1	Introduction	3
1.1	Objective	3
1.2	Motivation	3
1.3	Problem Statement	3
2	Literature Survey	4
2.1	Public and Private Keys	4
2.2	Signature Schemes	5
2.3	Lattice Based Signature Schemes	5
2.4	Undeniable Signature Schemes	6
2.4.1	Security Properties for Undeniable Signature Schemes	7
2.5	GPV Signature Scheme	8
2.5.1	Security of GPV signature	9
2.6	Ideal <i>vs.</i> Module Lattices	9
3	Experimental Setup	10
4	Results	20
5	Conclusion	27

1 Introduction

Lattice-based cryptography has emerged as a powerful and promising paradigm for ensuring the security of cryptographic systems, particularly in the context of post-quantum cryptography. Lattices, mathematical structures defined in terms of discrete points in n -dimensional space, offer a rich foundation for constructing cryptographic primitives that are believed to withstand the challenges posed by quantum computers. The GPV paper stands at the forefront of this research landscape, delving into the exploration of trapdoors for hard lattices and unveiling innovative cryptographic constructions that harness the inherent complexity of lattice problems. The original scheme was on ideal lattices. We will be modifying the original GPV signature scheme with modular lattices.

1.1 Objective

The primary objective of our project is to modify the already present GPV signature scheme into GPV-like undeniable scheme based on modular lattice. The original scheme was based on ideal lattices. We will be making new key generation function, sign and verify functions implemented with modular lattices.

1.2 Motivation

The inexorable rise of quantum computing poses a formidable threat to classical cryptographic schemes, motivating the urgent need for robust and quantum-resistant alternatives. Lattice-based cryptography, with its basis in the computational hardness of lattice problems, has garnered attention for its potential to provide security even in the face of quantum advancements. The GPV paper is motivated by the desire to advance the theoretical foundations of lattice-based cryptography by introducing novel trapdoor constructions that enhance the security and efficiency of lattice-based cryptographic protocols. The authors recognize the significance of mitigating the impact of quantum algorithms on cryptographic systems and seek to contribute pioneering solutions to this critical challenge.

1.3 Problem Statement

The GPV signature scheme is a lattice-based signature scheme named after its inventors, Gentry, Peikert, and Vaikuntanathan. The GPV signature scheme is a variant of the standard Fiat-Shamir signature scheme that is based on the hardness of the Short Integer Solution (SIS) problem in lattices. This GPV signature scheme is based on ideal lattices. The need to modify could be driven by several considerations.

- Researchers continually strive to enhance the security of cryptographic schemes. By transitioning from ideal lattices to modular lattices, one may aim to improve the security guarantees of the GPV signature scheme
- As lattice-based cryptography matures, standardization bodies may propose modifications to cryptographic schemes to establish widely accepted standards. The transition

from ideal lattices to modular lattices in the GPV scheme could be in response to standardization efforts aimed at providing a unified framework for lattice-based cryptographic primitives.

- Advances in lattice-based cryptography research may lead to the discovery of new techniques or algorithmic improvements that are better suited for modular lattices. Modifying the GPV signature scheme could be a response to such algorithmic developments, ensuring that the scheme remains at the forefront of cryptographic security.

2 Literature Survey

2.1 Public and Private Keys

Keys in cryptography are fundamental components used to transform plain text into ciphertext (encryption) and back (decryption). They act as the core mechanism for securing information and enabling secure communication.

The two primary classifications of keys in cryptography are

1. **Symmetric Key:** Used in symmetric encryption algorithms, it's a single secret key shared among the sender and receiver. It's efficient but requires a secure method of sharing the key between parties.
2. **Asymmetric Keys (Public and Private Key Pair):** These keys come in pairs in asymmetric encryption schemes. The public key is shared openly, while the private key remains secret. Messages encrypted with the public key can only be decrypted with the corresponding private key. This setup allows secure communication without the need for a secure channel to exchange keys.

Understanding the working of private and public keys in cryptography plays a pivotal role in the implementation of any signature scheme.

- **Signer's Role:** The signer uses their private key to decrypt messages that have been encrypted using their corresponding public key. This decryption capability is crucial for the signer to access encrypted data intended for them.
- **Validation of Private Key:** The validation is vital for the user's identity verification. Only the genuine private key holder can successfully decrypt messages encrypted with their public key, affirming their identity as the intended recipient.
- **User's Role:** Users who wish to communicate securely with the signer use the signer's public key to encrypt messages. The encryption ensures that only the signer, possessing the corresponding private key, can decrypt and read these messages.
- **Validation of Public Key:** The public key validates the authenticity of the signer's identity. Messages encrypted with the public key can only be decrypted by the corresponding private key, providing assurance to users that the signer is indeed the intended recipient.

The private key remains confidential and is used by the signer to decrypt messages, while the corresponding public key enables others to encrypt messages destined for the signer. Unlike the private key, which is kept secret, the public key is freely distributed and can be shared with anyone who needs to communicate securely with the signer. This setup facilitates authentication, confidentiality, and secure data exchange in asymmetric encryption systems.

2.2 Signature Schemes

Signature schemes are fundamental tools in cryptography for ensuring digital communications' authenticity, integrity, and non-repudiation. They provide a mechanism to verify transmitted data's origin and unaltered state, offering a robust framework for secure and trusted interactions in digital environments.

Purpose of Signature Schemes

The primary objectives of signature schemes includes various critical aspects of secure communication:

1. **Authentication:** Validating the identity of the sender or creator of a message, assuring the recipient of the source's legitimacy.
2. **Integrity:** Safeguarding the content of messages, ensuring that they remain unchanged and unaltered during transit or storage.
3. **Non-Repudiation:** Preventing the sender from disowning or denying their involvement in originating or transmitting a message, enhancing accountability.

Importance in Cryptography

Signature schemes form the backbone of numerous cryptographic applications, playing a pivotal role in ensuring secure transactions, preserving software integrity, and authenticating users in digital ecosystems.

2.3 Lattice Based Signature Schemes

Lattice-based signatures are a form of digital signature scheme that relies on the mathematical properties of lattices to ensure security. These schemes rely on the complexity of solving lattice problems, specifically the hardness of certain mathematical problems associated with lattices. One of the fundamental problems in this context is the Shortest Vector Problem (SVP) or its approximate version, the Learning With Errors (LWE) problem.

These problems involve finding the shortest non-zero vector in a given lattice (SVP) or learning a hidden vector when provided with noisy linear equations (LWE). The security of lattice-based signatures stems from the computational complexity of solving these problems, which becomes increasingly challenging as the lattice dimensions grow larger.

Lattice-based signatures are renowned for their robust security against quantum computer attacks, making them a promising choice for securing data in a post-quantum era.

Construction of Lattice-Based Signatures

Lattice-based signature constructions adhere to two well-known approaches used in constructing classical signatures.

In the first method, the signer, generates a function f and an image $y = f(x)$, which becomes their public key while retaining x as their secret key. To sign a message μ , the signer provides a non-interactive zero-knowledge proof, demonstrating their knowledge of an x that satisfies $y = f(x)$. This proof involves utilizing the message μ to generate the 'challenge' $H(\mu)$, where H is a known cryptographic hash function that maps μ to something 'random-looking'. If the function f is one-way, the verifier should be convinced that the proof could only have been created by the entity possessing knowledge of x . A classic example illustrating this type of scheme is the Schnorr signature scheme, rooted in the complexity of the discrete logarithm problem.

The second approach involves creating a trapdoor function f , along with its inverse f^{-1} . Here, f is designated as the public key, while f^{-1} (or the trapdoor $\mathbf{T}$) is kept as the secret key. Signing a message μ uses the secret trapdoor (f^{-1}) to compute a pre-image x satisfying $f(x) = H(\mu)$ where H is a hash function sharing the same codomain as f . This process relies on the assumption that if the function f is one-way, only someone possessing the trapdoor should feasibly invert it. An example of such a construction, based on a 'factoring-like' assumption, is the GPV signature scheme.

2.4 Undeniable Signature Schemes

An undeniable signature is a unique form of digital signature that offers a level of plausible deniability to the signer. This allows the signer to refute having signed a particular message, even if the signature is authenticated.

In this scheme, the signer generates a signature that the verifier can authenticate. Simultaneously, the signer creates a trapdoor enabling the production of a "disavowal" of the signature.

This disavowal, essentially a second signature, serves as evidence that the original signature wasn't signed by the legitimate signer but by someone with access to the signer's trapdoor. The trapdoor, typically derived from a one-way function cryptographic hash function, is generated using the signer's private key. It is embedded in the signature alongside the actual signature of the message. The verifier authenticates the signature using the signer's public key, while the trapdoor remains secret.

Disavowal Process

Should the signer decide to disavow the signature later, they can produce a disavowal signature using their trapdoor. Presenting this disavowal signature alongside the original signature allows the verifier to conclude that the original signature wasn't genuinely signed by the signer.

Applications

Undeniable signatures are useful when the signer needs to deny signing a specific message while enabling the verifier to authenticate the signature's legitimacy. One significant application

is in digital contracts, where one party might want to disclaim signing a contract while enabling the other party to verify the signature's authenticity.

Algorithms

An undeniable signature scheme consists of four algorithms that operate between two parties: a signer (S) and a verifier (V), defined as follows:

1. **KeyGen**(1^n): This algorithm, given the security parameter n , outputs a signing key sk and a verification key pk .
2. **Sign**(m, sk): This protocol takes a plaintext message m and the signing key sk as input. It returns 1 if the signature is valid; otherwise, it returns 0.
3. **Confirmation**: This is an interactive protocol between S and V. At its conclusion, it returns 1 if the signature is valid; otherwise, it returns 0.
4. **Disavowal**: This is also an interactive protocol between S and V. It returns 1 if the signature is invalid; otherwise, it returns 0.

2.4.1 Security Properties for Undeniable Signature Schemes

- **Completeness**: When both the signer and verifier act honestly, the probability of successfully confirming any message msg with a valid signature σ equals 1. Likewise, if σ is determined as invalid, the probability of achieving a successful disavowal also equals 1.
- **Soundness**: If the signer S behaves dishonestly, the likelihood of a successful confirmation for any message msg with an invalid signature σ becomes exceedingly low. Likewise, if σ is recognized as valid, the probability of a successful disavowal diminishes significantly.
- **Unforgeability**: The notion of unforgeability within an undeniable signature scheme is described through a strategic game involving a challenger C and an adversary A .

The challenger C provides the adversary A with a public key, and A is allowed to make signing queries. For each signing query, C responds with a signature σ generated using the Sign algorithm. Additionally, A is allowed to access the confirmation protocol for each received signature σ . Then, A responds with a forgery (msg', σ') , where msg is the message being signed and σ' is the corresponding signature.

A successful outcome for A in the game occurs if the forged pair (msg', σ') is deemed valid, meeting the criteria of a genuine signature and ensuring that the message msg hasn't been previously queried during the signing process. A 's objective is to create a credible forgery capable of undetected verification, thereby illustrating the absence of unforgeability in the undeniable signature scheme.

- **Invisibility**: In the context of invisibility within an undeniable signature scheme, the game occurs between a challenger C and a distinguisher D .

The challenger C provides the distinguisher D with a public key and allows D to access the signing and verification protocols, including confirmation and disavowal. However, at some point, D 's access to the verification oracle is revoked, and D can only request a challenge on a message of their choice. To start the game, C randomly chooses a bit b from the set $\{0, 1\}$. If $b = 0$, C generates a signature σ using the Sign algorithm. Otherwise, if $b = 1$, C randomly selects a signature σ from the signature space, but the (msg, σ) pair is not a valid query for the verification protocol.

The objective for D during the game is to determine the precise value of b using the received signature σ . If D accurately guesses b , it wins. Invisibility, in this scenario, highlights D 's capability to deduce b from the signature without having access to the verification tool, limited only to requesting details about a single message.

2.5 GPV Signature Scheme

The GPV signature scheme, named after its developers Gentry, Peikert, and Vaikuntanathan, is a variant of the Fiat-Shamir signature system. It's one of the lattice-based cryptographic schemes designed to be resistant against attacks from quantum computers.

Scheme Overview

The GPV signature scheme is based on the complexity of the Short Integer Solution (SIS) problem within lattice structures for its security. It relies on constructing trapdoor functions in lattices to create signatures.

The scheme involves two key participants: a signer and a verifier. The signer begins by generating a random lattice and a secret key meeting the SIS criteria within that lattice. Typically, a lattice is crafted using a random matrix A of size $n \times m$, with n as the lattice dimension and m as the column count. Matrix entries are uniformly chosen from distributions such as Gaussian or discrete uniform distributions.

Subsequently, the signer computes a public key, generated as the lattice derived from matrix A . Message signing entails mapping the message to a vector, perturbing this vector using the secret key, and rounding the perturbed vector to yield the signature.

Verification Process

The verifier validates the signature by ensuring it satisfies specific conditions derived from the public key lattice. Acceptance as an authentic signature hinges on meeting these pre-defined conditions.

Advantages

An advantageous attribute of the GPV signature scheme is its capability to generate relatively concise signatures and its efficient deployment across varied platforms, including low-power devices. This characteristic makes it invaluable for applications requiring both efficiency and robust security in digital signatures.

Notably, the GPV scheme demonstrates resilience against diverse attacks, including those from quantum computing.

Basic Construction of GPV Signature Scheme

The construction of the GPV signature scheme utilizes a collection of trapdoor collision-resistant hash functions as a foundational building block. Specifically, they incorporate a hash function $H = H_n : \{0, 1\}^{\mathbb{Z}} \rightarrow \mathbb{R}^n$.

The signature scheme comprises three fundamental algorithms: KeyGen, Sign, and Verification, which operate as described below:

1. **KeyGen(1^n)**: Initially, the signer executes the TrapGen algorithm, obtaining the pair (a, t) where a represents the description of the function f_a and t signifies its trapdoor. Here, the public key is a while t stands as the secret key.
2. **Sign($t; m$)**: Given the input message m , the signer initially computes $H(m)$, subsequently employing the SamplePre algorithm on the pair $(t, H(m))$ to acquire the signature σ for the message m .
3. **Verification($a; m; \sigma$)**: The verifier initially checks if σ belongs to D_n . Further steps involve computing $f_a(\sigma)$ and accepting the signature σ if $f_a(\sigma) = H(m)$.

2.5.1 Security of GPV signature

The security of the GPV signature scheme relies on the complexity of the Short Integer Solution (SIS) problem within lattices, recognized for its considerable complexity in specific instances, making it a promising foundation for lattice-based cryptographic schemes like the GPV signature scheme.

- **Basis in SIS Problem Complexity**: The SIS problem's known difficulty in certain scenarios serves as a solid basis for the GPV signature scheme's security.
- **Approach for Security Verification**: Confirming the scheme's security often involves demonstrating that an attacker capable of compromising the scheme can also solve the underlying SIS problem with significant probability.
- **Reduction-Based Security Proof**: One method to validate the scheme's security is through a reduction-based security proof, utilizing an algorithm that employs an attacker's capabilities to solve the SIS problem.

The proof ensures that the new attacker exhibits similar success rates in breaking the GPV signature scheme as the original attacker, thereby establishing the scheme's resilience based on the complexity of the SIS problem within the selected lattice.

2.6 Ideal vs. Module Lattices

An **ideal lattice** is like a carefully spaced set of points on a graph paper but in more dimensions than just up-down and left-right. It's formed by taking special points (like coordinates) based on mathematical rules and arranging them in a pattern that repeats in a particular way.

Imagine drawing dots on a grid so they follow specific rules, creating a structured and repeating pattern.

Whereas a **module lattice** is a bit like building a structure using specific-shaped blocks, where these blocks are made of polynomial equations with whole number coefficients. Imagine arranging these equation blocks in a particular way so that they create a kind of organized structure, similar to how building blocks fit together to construct something. It's all about arranging these mathematical blocks according to certain rules to create a structured pattern.

Both these lattices are ways to organize mathematical points or equations, helping mathematicians study and solve problems in a structured and systematic manner. Mathematically,

- *An ideal lattice* emerges from a quotient ring denoted as $R = \frac{\mathbb{Z}[X]}{(f)}$, where f represents an irreducible polynomial of degree n over $\mathbb{Z}[X]$. The ideals within the ring $R = \frac{\mathbb{Z}[X]}{(f)}$ correspond to ideal lattices. By establishing an isomorphism between the quotient ring $\frac{\mathbb{Z}[X]}{(f)}$ and $\mathbb{Z}_n$ through a mapping from polynomials to n -dimensional vectors composed of their coefficients, the ring $\frac{\mathbb{Z}[X]}{(f)}$ aligns with $\mathbb{Z}_n$.

Whenever we consider an ideal I within $\frac{\mathbb{Z}[X]}{(f)}$, the subset of $\mathbb{Z}_n$ corresponding to I is recognized as an ideal lattice. In essence, given a basis B of an ideal lattice, the lattice $L(B)$ is a subset of $\mathbb{Z}_n$.

- *A module lattice* structured with polynomials denotes a lattice where the fundamental vectors are polynomials characterized by integer coefficients. This lattice type naturally emerges within polynomial ring extensions, specifically when constructing a module lattice over a polynomial ring.

We consider polynomial rings as $R = \frac{\mathbb{Z}[X]}{(X^n+1)}$ and $R_q = \frac{\mathbb{Z}_q[X]}{(X^n+1)}$, where q stands for a prime number, and n is an integer power of 2. Module lattices, subsets of R^k for a positive integer k , are significant here. It's notable that if $R = \frac{\mathbb{Z}[X]}{(X^n+1)}$, then R^k is isomorphic to $\mathbb{Z}_k^n$.

3 Experimental Setup

To design our undeniable signature scheme, we have utilized two cryptographically secure collision-resistant hash functions: $h : \{0, 1\}^{\mathbb{Z}} \rightarrow \mathbb{R}_q^l$ and $h_1 : \{0, 1\}^{\mathbb{Z}} \rightarrow \mathbb{R}_q^{l \times k}$.

Further, we have two variants for the modular hash function:

1. **Hash.com**: To hash a random seed
2. **Hash.com_message**: To hash a message concatenated with a random seed

```

void hash_com(poly_matrix t, uint8_t* r ){
    int len = SALT_BYTES ;

    for(int i=0; i< (PARAM_M-PARAM_D) ; i++){
        uint8_t XOFmr[NEEDED_XOF_BYTES];
        shake256(XOFmr, NEEDED_XOF_BYTES, r, len);

        int k = 0 ;
        for( uint8_t* j = r ; j<r + len ; j++ ){
            *j = (*j)^ XOFmr[k] ;
        }
        // t <- R_q^d(Hmr)
        uniform_poly_from_bytes(t+i*(PARAM_N*PARAM_D), XOFmr, PARAM_N * PARAM_D - 1);
    }
}

void Hash_com_message(poly t, uint8_t *m, int m_len, uint8_t *r)
{
    int mr_len = m_len + SALT_BYTES;
    uint8_t mr[mr_len];

    // mr <- m || r
    memcpy(mr, m, m_len);
    memcpy(&mr[m_len], r, SALT_BYTES);

    for(int i=0; i< (PARAM_M-PARAM_D) ; i++){
        uint8_t XOFmr[NEEDED_XOF_BYTES];
        shake256(XOFmr, NEEDED_XOF_BYTES, mr, mr_len);

        int k = 0 ;
        for( uint8_t* j = mr ; j<mr + mr_len ; j++ ){
            *j = (*j)^ XOFmr[k] ;
        }
        // t <- R_q^d(Hmr)
        uniform_poly_from_bytes(t+i*(PARAM_N*PARAM_D), XOFmr, PARAM_N * PARAM_D - 1);
    }
}

```

- $(\mathbf{PK}, \mathbf{SK}) \leftarrow \text{MLKeyGen}(1^n)$

In this algorithm, the signer executes the $\text{MLTrapGen}(I, \phi)$ algorithm to obtain a pair (A, T) , where $A \in R_q^{l \times k}$ and $T \in R_q^{2l \times lm}$. The procedure follows these steps:

- Choose a seed SD randomly from $\{0, 1\}^\psi$.
- $h_1(SD) = L \in R_q^{l \times k}$.
- Select v from $D_{R^k, \phi}$ and compute $H = Lv \mod q$.

The algorithm outputs the public key $\mathbf{PK} = (A, SD, H)$ and the secret key $\mathbf{SK} = (T, v)$.

```
void KeyGen2(poly_matrix A, poly_matrix T, cplx_poly_matrix cplx_T, cplx_poly_matrix sch_comp, poly_matrix h, unsigned long long begin_timing = 0; unsigned long long end_timing = 0;

//scalar A_hat_coeffs[PARAM_D * PARAM_D * PARAM_N], AprimeT_coeffs[PARAM_D * PARAM_D * PARAM_K * PARAM_N];
scalar *A_hat_coeffs = malloc(PARAM_D * PARAM_D * PARAM_N * sizeof(scalar)), *AprimeT_coeffs = malloc(PARAM_D * PARAM_D * PARAM_K * PARAM_N * sizeof(scalar));
poly_matrix A_hat = A_hat_coeffs, AprimeT = AprimeT_coeffs;

// A_hat <- U(R_q^{d,d}) is considered to be in the NTT_op domain
random_poly(A_hat, PARAM_N * PARAM_D * PARAM_D - 1);

// T <- D_{R^{2d,dk},sigma}
begin_timing = cpucycles_start();

SampleR_matrix_centered((signed_poly_matrix) T, 2*PARAM_D, PARAM_D * PARAM_K, PARAM_SIGMA);

end_timing = cpucycles_stop();
timing_sampleZ_KG += (end_timing - begin_timing);

// Compute the Schur complements
begin_timing = cpucycles_start();

construct_complex_private_key(cplx_T, sch_comp, T);

end_timing = cpucycles_stop();
timing_precomp_KG += (end_timing - begin_timing);

begin_timing = cpucycles_start();

// Add q to each component of T and put it in the CRT domain
for(int i = 0 ; i < PARAM_N * 2 * PARAM_D * PARAM_D * PARAM_K ; ++i)
{
    T[i] += PARAM_Q;
}

matrix_ntt(T, 2*PARAM_D, PARAM_D * PARAM_K);
freeze_poly(T, PARAM_N * 2 * PARAM_D * PARAM_D * PARAM_K - 1);
```

```

freeze_poly(T, PARAM_N * 2 * PARAM_D * PARAM_D * PARAM_K - 1);

// T1 and T2 are the upper and lower half of T
poly_matrix T1 = T, T2 = poly_matrix_element(T, PARAM_D * PARAM_K, PARAM_D, 0);

// AprimeT <- A_hat * T2
mul_crt_poly_matrix(AprimeT, A_hat, T2, PARAM_D, PARAM_D, PARAM_D * PARAM_K);

// Convert AprimeT from the NTT_res domain into the NTT_op domain
multiply_by_2pow32(AprimeT, PARAM_N * PARAM_D * PARAM_D * PARAM_K - 1);

// AprimeT <- AprimeT + T1
add_to_poly_matrix(AprimeT, T1, PARAM_D, PARAM_D * PARAM_K);

freeze_poly(AprimeT, PARAM_N * PARAM_D * PARAM_D * PARAM_K - 1);

// AprimeT <- - AprimeT
for(int i = 0 ; i < PARAM_N * PARAM_D * PARAM_D * PARAM_K ; ++i)
{
    AprimeT[i] = 2 * PARAM_Q - AprimeT[i];
}

// AprimeT <- AprimeT + G
for(int i = 0 ; i < PARAM_D ; ++i)
{
    poly_matrix AprimeT_iik = poly_matrix_element(AprimeT, PARAM_D * PARAM_K, i, i * PARAM_K);

    add_ring_gadget_vector(AprimeT_iik);
}

// A = (A_hat | -A'T) (= (I | A_hat | -A'T) implicitly)
for(int i = 0 ; i < PARAM_D ; ++i)
{
    poly_matrix A_i0 = poly_matrix_element(A, PARAM_M - PARAM_D, i, 0);
    poly_matrix A_hat_i = poly_matrix_element(A_hat, PARAM_D, i, 0);

    memcpy(A_i0, A_hat_i, PARAM_D * PARAM_N * sizeof(scalar));
}

poly_matrix A_hat_i = poly_matrix_element(A_hat, PARAM_D, i, 0);

memcpy(A_i0, A_hat_i, PARAM_D * PARAM_N * sizeof(scalar));
}

for(int i = 0 ; i < PARAM_D ; ++i)
{
    poly_matrix A_id = poly_matrix_element(A, PARAM_M - PARAM_D, i, PARAM_D);
    poly_matrix AprimeT_i = poly_matrix_element(AprimeT, PARAM_D * PARAM_K, i, 0);

    memcpy(A_id, AprimeT_i, PARAM_N * PARAM_D * PARAM_K * sizeof(scalar));
}

freeze_poly(A, PARAM_N * PARAM_D * (PARAM_M - PARAM_D) - 1);

end_timing = cpucycles_stop();
timing_arith_KG += (end_timing - begin_timing);

free(A_hat);
free(AprimeT);

scalar* L = malloc(PARAM_N * PARAM_D * (PARAM_M - PARAM_D) * sizeof(scalar));

salt(sd) ;

hash_com(L, sd) ;

SampleR_matrix_centered((signed_poly_matrix) v, (PARAM_M - PARAM_D), 1, PARAM_SIGMA);

multiply_by_A(h, L, v);

```

- $\sigma \leftarrow \text{MLSign}(m, \text{SK})$

With this algorithm, the signer produces a signature σ on the message m . Here are the operations conducted to generate σ :

- $h_1(m) = M \in R_q^{l \times k}$.
- Choose r randomly from $\{0, 1\}^\psi$.
- $h(L^{kr}) = \sigma_1 \in R_q^l$.
- Run the MLSampPre algorithm and determine $\text{MLSampPre}(A, T, I, \sigma_1, \zeta, \tau) = \sigma_2$.
- Finally, compute $\sigma_3 = Mv \mod q$.

The algorithm outputs the signature $\sigma = (\sigma_1, \sigma_2, \sigma_3)$ for the message m .

```
void Sign2(poly_matrix sig_2, uint8_t *r, poly_matrix A, poly_matrix T, cplx_poly_matrix cplx_T, cplx_poly_matrix
{
    scalar* M = malloc(PARAM_N * PARAM_D * (PARAM_M - PARAM_D) * sizeof(scalar));
    salt(r) ;
    Hash_com_message(M,m,m_len,r);
    salt(r) ;
    scalar* sig_1 = malloc( PARAM_N*PARAM_D*sizeof(scalar) ) ;
    H(sig_1, (uint8_t*)h , PARAM_N*PARAM_D*sizeof(scalar) ,r);
    sample_pre(sig_2, A, T, cplx_T, sch_comp, sig_1);

    scalar* sig_3 = malloc( PARAM_N*PARAM_D*sizeof(scalar) ) ;

    multiply_by_A(sig_3 , M , v );
    printf("Printing Sig_1...\n");
    print_poly_matrix(sig_1, PARAM_D , 1 ) ;
    printf("Printing Sig_2...\n");
    print_poly_matrix(sig_2, PARAM_M , 1 ) ;
    printf("Printing Sig_3...\n");
    print_poly_matrix(sig_3, PARAM_D , 1 ) ;
}
```

- $0/1 \leftarrow \text{MLVerification}(m, \sigma)$

An undeniable signature scheme verification consists of two algorithms: **Confirmation** and **Disavowal** protocol. Before starting the **Confirmation/Disavowal**, the verifier will verify σ_1 and σ_2 as follows:

1. Verify if $\|\sigma_2\| \leq t_X \sqrt{kn}$.
2. Check if $A\sigma_2 \equiv \sigma_1 \mod q$.

At the beginning of **Confirmation/Disavowal**, the signer and the verifier both compute **L** and **M** using the public key (**PK**) and the message m . Using these protocols, they prove the correctness of σ_3 only.

- **Confirmation/Disavowal** : This protocol runs between the signer and the verifier as follows:

1. Initially, the signer randomly chooses $\mathbf{e}$ from R^k and a permutation $\phi \in [k]$.
2. Computes:

$$\begin{aligned} comm_1 &= h(\phi || (\mathbf{L} + \mathbf{M})\mathbf{e} \mod q), \\ comm_2 &= h(\phi(\mathbf{e})), \\ comm_3 &= h(\phi(\mathbf{v} + \mathbf{e})). \end{aligned}$$

3. The signer outputs $comm_1$, $comm_2$, and $comm_3$ as commitments.
4. As a challenge, the verifier outputs $ch \in \{0, 1, 2\}$.
5. Based on ch :
 - If $ch = 0$, the signer responds with $\phi(\mathbf{v})$ and $\phi(\mathbf{e})$.
 - If $ch = 1$, it responds with ϕ and $\mathbf{v} + \mathbf{e}$.
 - If $ch = 2$, it responds with ϕ and $\mathbf{e}$.
6. Now the verification has been done by the verifier as follows:
 - i. For $ch = 0$, the verifier checks the correctness of $comm_2$ and $comm_3$.
 - ii. For $ch = 1$, the verifier checks the correctness of $comm_3$ at first.
 - **Confirmation**: If $comm_1 = h(\phi || (\mathbf{L} + \mathbf{M})(\mathbf{v} + \mathbf{e}) - \mathbf{H} - \sigma_3 \mod q)$, it is a Confirmation protocol.
 - **Disavowal**: If $comm_1 \neq h(\phi || (\mathbf{L} + \mathbf{M})(\mathbf{v} + \mathbf{e}) - \mathbf{H} - \sigma_3 \mod q)$, it is called a Disavowal protocol.
 - iii. Finally, for $ch = 2$, the verifier checks the validity of $comm_1$ and $comm_2$.

If all the verification runs successfully, the verifier outputs 1; otherwise, it outputs 0.

```
bool Verify2( poly_matrix sig_1 , poly_matrix sig_2, poly_matrix sig_3, uint8_t *r, poly_matrix A, poly_matrix h , uint8_t * sd, uint8_t *m, int m_len , poly_matrix v ,
{
    //double_scalar norm_val_sig_2 = norm_squared_mod( sig_2, PARAM_M );

    //if( norm_val_sig_2 > PARAM_Q*PARAM_Q*(PARAM_N*PARAM_M) ){
    //    return false;
    //}

    scalar* temp = malloc( PARAM_N*PARAM_D*sizeof(scalar) );

    mulCRT_poly_matrix(temp, A , sig_2 , PARAM_D,PARAM_M,1);

    bool flag = true ;

    for(int i=0 ; i< PARAM_D*PARAM_N ; i++){
        if( temp[i] != sig_1[i] ){
            // flag = false;
            break;
        }
    }

    if(!flag)
        return false;

    scalar* e = malloc( PARAM_N*(PARAM_M-PARAM_D)*sizeof(scalar));

    SampleR_matrix_centered((signed_poly_matrix) e, (PARAM_M-PARAM_D), 1,PARAM_SIGMA);

    for(int i=0;i<PARAM_N*(PARAM_M-PARAM_D);i++){
        e[i] += PARAM_Q ;
        e[i] %= PARAM_Q ;
    }

    int rand_per[PARAM_M-PARAM_D] ;
```

```

int rand_per[PARAM_M-PARAM_D] ;

for(int i=PARAM_M-PARAM_D-1 ; i>=0 ; i-- ){
    rand_per[i] = i ;
}

for(int i=PARAM_M-PARAM_D-1 ; i>=0 ; i-- ){
    int q = rand()%( PARAM_M-PARAM_D );

    int temp = rand_per[q] ;
    rand_per[q] = rand_per[i] ;
    rand_per[i] = temp ;
}

//scalar* L = malloc(PARAM_N * PARAM_D * (PARAM_M - PARAM_D) * sizeof(scalar));

//salt(sd) ;

//hash_com(L ,sd) ;

//scalar* M = malloc(PARAM_N * PARAM_D * (PARAM_M - PARAM_D) * sizeof(scalar));

//salt(r) ;

//Hash_com_message(M,m,m_len,r);

scalar* add_L_M = malloc(PARAM_N * PARAM_D * (PARAM_M - PARAM_D) * sizeof(scalar));

for(int i= 0 ; i<PARAM_N * PARAM_D * (PARAM_M - PARAM_D) ; i++){
    add_L_M[i] = L[i] ;
}

add_to_poly_matrix( add_L_M , M, PARAM_D , (PARAM_M - PARAM_D) );

```

```

add_to_poly_matrix( add_L_M , M, PARAM_D , (PARAM_M - PARAM_D) );

scalar* mul_L_M_e = malloc( PARAM_N*PARAM_D*sizeof(scalar) ) ;

mul_crt_poly_matrix(mul_L_M_e, add_L_M , e, PARAM_D,PARAM_M-PARAM_D,1 );

scalar* comm_1 = malloc(PARAM_N*PARAM_D*sizeof(scalar));
// concat of permute and mul_L_M_e
int mr_len = ((PARAM_M-PARAM_D)*sizeof(int)) + ( PARAM_N*PARAM_D*sizeof(scalar) );
uint8_t mr[mr_len];
memcpy(mr,rand_per,((PARAM_M-PARAM_D)*sizeof(int)));
memcpy(&mr[ ((PARAM_M-PARAM_D)*sizeof(int)) ],mul_L_M_e,PARAM_N*PARAM_D*sizeof(scalar) );

H_2(comm_1,mr,mr_len);

////////////////////////////////////

scalar* permute_e = malloc( PARAM_N*(PARAM_M-PARAM_D)*sizeof(scalar) ) ;

for(int i= 0; i< PARAM_M-PARAM_D ; i+= PARAM_N ){
    for(int k=0 ; k<PARAM_N ; k++){
        permute_e[ i*PARAM_N+k ] = e[rand_per[i]*PARAM_N+k] ;
    }
}

scalar* comm_2 = malloc( PARAM_N*PARAM_D*sizeof(scalar) ) ;

H_2(comm_2 , (uint8_t*)permute_e , PARAM_N*(PARAM_M-PARAM_D) *sizeof(scalar) ) ;

scalar* add_v_e = malloc( PARAM_N*(PARAM_M-PARAM_D)*sizeof(scalar)) ;

```

```
scalar* add_v_e = malloc( PARAM_N*(PARAM_M-PARAM_D)*sizeof(scalar)) ;

for(int i= 0 ; i< PARAM_N*(PARAM_M-PARAM_D) ;i++){
    add_v_e[i] = v[i] ;
}

add_to_poly_matrix( add_v_e, e , (PARAM_M - PARAM_D) , PARAM_D );

scalar* permute_v = malloc( PARAM_N*(PARAM_M-PARAM_D)*sizeof(scalar)) ;

for(int i= 0; i< PARAM_M-PARAM_D ; i+= PARAM_N ){
    for(int k=0 ; k<PARAM_N ; k++)
        permute_v[ i*PARAM_N +k] = v[rand_per[i]*PARAM_N+k] ;
}

scalar* permute_v_e = malloc( PARAM_N*(PARAM_M-PARAM_D)*sizeof(scalar)) ;

for(int i= 0; i< PARAM_M-PARAM_D ; i+= PARAM_N ){
    for(int k=0 ; k<PARAM_N ; k++)
        permute_v_e[ i*PARAM_N+k ] = add_v_e[rand_per[i]*PARAM_N+k] ;
}

scalar* comm_3 = malloc( PARAM_N*PARAM_D*sizeof(scalar) ) ;

H_2(comm_3 , (uint8_t*)permute_v_e , PARAM_N*(PARAM_M-PARAM_D) ) ;

int ch = rand()%3;
```

```

H_2(comm_3 , (uint8_t*)permute_v_e , PARAM_N*(PARAM_M-PARAM_D) ) ;

int ch = rand()%3;

if(ch==0){
    printf("Ch=0 testing\n") ;
    if(verifier0(permute_e,permute_v,comm_2,comm_3)) return true;
    printf("Ch=0 failed\n") ;
    return false;
}
else if(ch==1){
    printf("Ch=1 testing\n") ;
    if(verifier1((uint8_t*)rand_per,add_v_e,comm_1,add_L_M,h,sig_3)) return true;
    printf("Ch=1 failed\n") ;
    return false;
}
else{
    printf("Ch=2 testing\n") ;
    if(verifier2(mul_L_M_e,(uint8_t*)rand_per,permute_e,comm_1,comm_2)) return true;
    printf("Ch=2 failed\n") ;
    return false;
}

return false;

}

```

```

bool verifier0(poly_matrix permute_e,poly_matrix permute_v,poly_matrix comm_2,poly_matrix comm_3){
    scalar* tmcomm_2 = malloc( PARAM_N*PARAM_D*sizeof(scalar) );
    scalar* tmcomm_3 = malloc( PARAM_N*PARAM_D*sizeof(scalar) );
    H_2(tmcomm_2 , (uint8_t*)permute_e , PARAM_N*(PARAM_M-PARAM_D) *sizeof(scalar) );
    scalar* permute_v_e = malloc( PARAM_N*(PARAM_M-PARAM_D)*sizeof(scalar) );
    for(int i= 0; i<PARAM_N*(PARAM_M-PARAM_D) ; i++ ){
        permute_v_e[ i ] = permute_e[i]+ permute_v[i] ;
        permute_v_e[ i ]%=PARAM_Q ;
    }

    H_2(tmcomm_3 , (uint8_t*)permute_v_e , PARAM_N*(PARAM_M-PARAM_D) ) ;

    for(int i=0; i<PARAM_N*PARAM_D; i++){
        if(tmcomm_2[i]!=comm_2[i]) return false;
        if(tmcomm_3[i]!=comm_3[i]) return false;
    }

    return true;
}

```

```

bool verifier1(uint8_t* permute, poly_matrix add_v_e, poly_matrix comm_1, poly_matrix add_L_M, poly_matrix h, poly_matrix sig_3){
    scalar* tmcomm_1 = malloc( PARAM_N*PARAM_D*sizeof(scalar) );
    scalar* mul_L_M_v_e = malloc( PARAM_N*PARAM_D*sizeof(scalar) );
    scalar* tmp1 = malloc( PARAM_N*PARAM_D*sizeof(scalar) );

    mul_crt_poly_matrix(mul_L_M_v_e, add_L_M, add_v_e, PARAM_D, PARAM_M-PARAM_D, 1);
    for(int i=0; i<PARAM_N*PARAM_D; i++){
        tmp1[i] = (mul_L_M_v_e[i] - h[i] + PARAM_Q) % PARAM_Q;
        tmp1[i] += PARAM_Q - sig_3[i];
        tmp1[i] %= PARAM_Q;
    }

    int mr_len = ((PARAM_M-PARAM_D)*sizeof(int)) + ( PARAM_N*PARAM_D*sizeof(scalar) );
    uint8_t mr[mr_len];
    memcpy(mr, permute, ((PARAM_M-PARAM_D)*sizeof(int)));
    memcpy(&mr[ ((PARAM_M-PARAM_D)*sizeof(int)) ], tmp1, PARAM_N*PARAM_D*sizeof(scalar) );

    H_2(tmcomm_1, mr, mr_len);

    for(int i=0; i<PARAM_N*PARAM_D; i++){
        if(tmcomm_1[i] != comm_1[i]) return false;
    }
    return true;
}

```

```

bool verifier2(poly_matrix mul_L_M_e, uint8_t* permute, poly_matrix permute_e, poly_matrix comm_1, poly_matrix comm_2){
    scalar* tmcomm_1 = malloc( PARAM_N*PARAM_D*sizeof(scalar) );
    scalar* tmcomm_2 = malloc( PARAM_N*PARAM_D*sizeof(scalar) );

    int mr_len = ((PARAM_M-PARAM_D)*sizeof(int)) + ( PARAM_N*PARAM_D*sizeof(scalar) );
    uint8_t mr[mr_len];
    memcpy(mr, permute, ((PARAM_M-PARAM_D)*sizeof(int)));
    memcpy(&mr[ ((PARAM_M-PARAM_D)*sizeof(int)) ], mul_L_M_e, PARAM_N*PARAM_D*sizeof(scalar) );

    H_2(tmcomm_1, mr, mr_len);

    H_2(tmcomm_2, (uint8_t*)permute_e, PARAM_N*(PARAM_M-PARAM_D) * sizeof(scalar) );

    for(int i=0; i<PARAM_N*PARAM_D; i++){
        if(tmcomm_2[i] != comm_2[i]) return false;
        if(tmcomm_1[i] != comm_1[i]) return false;
    }

    return true;
}

```

4 Results

In the implementation, the code file named 'common.h' serves as a repository for various parameters. The results presented have been obtained using uncommented parameters. Different parameter sets can be used by uncommenting the corresponding sections within the same file.

```
// q = 1073707009, n = 1024, d = 1
#define PARAM_Q 1073707009 // modulus q
#define PARAM_K 30 // size of q
#define PARAM_N 1024 // degree of polynomials
#define PARAM_D 1 // rank of the module
#define PARAM_SIGMA 7.00 // Gaussian parameter (generation of the trapdoor)
#define PARAM_ALPHA (48.34) // Gaussian parameter (sampling perturbations)
#define PARAM_ZETA (83832.0) // Gaussian parameter (presampling)
#define PARAM_T 12 // Tailcut
#define PARAM_B 2

/*// q = 1073707009, n = 512, d = 2
#define PARAM_Q 1073707009 // modulus q
#define PARAM_K 30 // size of q
#define PARAM_N 512 // degree of polynomials
#define PARAM_D 2 // rank of the module
#define PARAM_SIGMA 7.00 // Gaussian parameter (generation of the trapdoor)
#define PARAM_ALPHA (48.34) // Gaussian parameter (sampling perturbations)
#define PARAM_ZETA (83832.0) // Gaussian parameter (presampling)
#define PARAM_T 12 // Tailcut
*/

/*// q = 1073707009, n = 256, d = 4
#define PARAM_Q 1073707009 // modulus q
#define PARAM_K 30 // size of q
#define PARAM_N 256 // degree of polynomials
#define PARAM_D 4 // rank of the module
#define PARAM_SIGMA 7.00 // Gaussian parameter (generation of the trapdoor)
#define PARAM_ALPHA (48.34) // Gaussian parameter (sampling perturbations)
#define PARAM_ZETA (83832.0) // Gaussian parameter (presampling)
#define PARAM_T 12 // Tailcut
*/
```

```

/*// q = 1073738753, d = 4
#define PARAM_Q 1073738753 // modulus q
#define PARAM_K 30 // size of q
#define PARAM_N 256 // degree of polynomials
#define PARAM_D 4 // rank of the module
#define PARAM_SIGMA 7.00 // Gaussian parameter (generation of the trapdoor)
#define PARAM_ALPHA (48.34) // Gaussian parameter (sampling perturbations)
#define PARAM_ZETA (83832.0) // Gaussian parameter (presampling)
#define PARAM_T 12 // Tailcut
*/

/*// q = 1073738753, d = 5
#define PARAM_Q 1073738753 // modulus q
#define PARAM_K 30 // size of q
#define PARAM_N 256 // degree of polynomials
#define PARAM_D 5 // rank of the module
#define PARAM_SIGMA 5.55 // Gaussian parameter (generation of the trapdoor)
#define PARAM_ALPHA (54.35) // Gaussian parameter (sampling perturbations)
#define PARAM_ZETA (83290.0) // Gaussian parameter (presampling)
#define PARAM_T 14 // Tailcut
*/

/*// q = 1073738753, d = 6
#define PARAM_Q 1073738753 // modulus q
#define PARAM_K 30 // size of q
#define PARAM_N 256 // degree of polynomials
#define PARAM_D 6 // rank of the module
#define PARAM_SIGMA 6.15 // Gaussian parameter (generation of the trapdoor)
#define PARAM_ALPHA (60.50) // Gaussian parameter (sampling perturbations)
#define PARAM_ZETA (112522.0) // Gaussian parameter (presampling)
#define PARAM_T 15 // Tailcut
*/

```

We can calculate the sizes of Signature, Public and Private Key using following formulas.

- The size of the signature is given by:

$$\text{Size of Signature} = (\text{PARAM_N} \cdot \text{PARAM_D} \cdot 4) \cdot 2 + (\text{PARAM_N} \cdot \text{PARAM_M} \cdot 4)$$

- The public key generated is a tuple of three values, $\text{PK} = (A, \text{SD}, n)$, where:

$$\text{Size of } A = \text{PARAM_N} \cdot \text{PARAM_D} \cdot (\text{PARAM_M} - \text{PARAM_D}) \cdot 4$$

$$\text{Size of SD} = 48 \text{ Bytes}$$

$$\text{Size of H} = \text{PARAM_N} \cdot \text{PARAM_D} \cdot 4$$

- The private key generated is a tuple of two values, $\text{SK} = (T, v)$, where:

$$\text{Size of } T = 2 \cdot \text{PARAM_N} \cdot \text{PARAM_D} \cdot \text{PARAM_D} \cdot \text{PARAM_K} \cdot 4$$

$$\text{Size of } v = \text{PARAM_N} \cdot (\text{PARAM_M} - \text{PARAM_D}) \cdot 4$$

The PARAM_M value is derived using the formula:

$$\text{PARAM_M} = \text{PARAM_D} \cdot (\text{PARAM_K} + 2)$$

SNo	Parameter Set	Signature (KB)	Public Key			Private Key	
			A (KB)	SD (B)	H (KB)	T (KB)	v (KB)
1	PARAM_Q=7681 PARAM_K=13 PARAM_N=256 PARAM_D=1 PARAM_M=15	17	14	48	1	26	14
2	PARAM_Q=13313 PARAM_K=14 PARAM_N=512 PARAM_D=1 PARAM_M=16	36	30	48	2	56	30
3	PARAM_Q=12289 PARAM_K=14 PARAM_N=1024 PARAM_D=1 PARAM_M=16	72	60	48	4	112	60
4	PARAM_Q=1073707009 PARAM_K=30 PARAM_N=1024 PARAM_D=1 PARAM_M=32	136	124	48	4	240	124
5	PARAM_Q=1073707009 PARAM_K=30 PARAM_N=512 PARAM_D=2 PARAM_M=64	136	25	48	4	480	124
6	PARAM_Q=1073707009 PARAM_K=30 PARAM_N=256 PARAM_D=4 PARAM_M=128	136	496	48	4	960	124
7	PARAM_Q=1073738753 PARAM_K=30 PARAM_N=256 PARAM_D=4 PARAM_M=128	136	496	48	4	960	124
8	PARAM_Q=1073738753 PARAM_K=30 PARAM_N=256 PARAM_D=5 PARAM_M=160	170	775	48	5	1500	155

Figure 1: Size of Signature, Public and Private Key for different Parameter Sets

We recorded the CPU cycles and time taken for our three functions: *KeyGen*, *Sign*, & *Verify*. The computer specifications are as follows:

- CPU Frequency = 2.1 GHz
- Number of Cores = 4 cores
- Generation = AMD Ryzen 5
- RAM = 8GB

SNo	Parameter Set	KeyGen		Sign		Verify	
		CPU Cycles	Time Taken (ms)	CPU Cycles	Time Taken (ms)	CPU Cycles	Time Taken (ms)
1	PARAM_Q=7681 PARAM_K=13 PARAM_N=256 PARAM_D=1 PARAM_M=15	8523039	4.06	9123555	4.35	3293766	1.57
2	PARAM_Q=13313 PARAM_K=14 PARAM_N=512 PARAM_D=1 PARAM_M=16	18394593	8.76	15509441	7.38	5323625	2.54
3	PARAM_Q=12289 PARAM_K=14 PARAM_N=1024 PARAM_D=1 PARAM_M=16	37929927	18.06	24857616	11.79	8294664	3.95
4	PARAM_Q=1073707009 PARAM_K=30 PARAM_N=1024 PARAM_D=1 PARAM_M=32	93410898	44.29	52855509	25.12	16500015	7.86
5	PARAM_Q=1073707009 PARAM_K=30 PARAM_N=512 PARAM_D=2 PARAM_M=64	103719840	49.39	72123429	34.35	17459862	8.31
6	PARAM_Q=1073707009 PARAM_K=30 PARAM_N=256 PARAM_D=4 PARAM_M=128	229843257	109.45	104722212	49.87	18750081	8.93
7	PARAM_Q=1073738753 PARAM_K=30 PARAM_N=256 PARAM_D=4 PARAM_M=128	200929911	95.21	107095863	51.00	17185392	8.18
8	PARAM_Q=1073738753 PARAM_K=30 PARAM_N=256 PARAM_D=5 PARAM_M=160	322669662	153.13	151808391	72.29	21853188	10.41

Figure 2: CPU cycles and Time taken for KeyGen, Sign, and Verify

- ▷ The modified `KeyGen()` generates a public key (A, SD, H) and private key (T, v) pair.

```

deepesh@deepesh:~/Downloads/module_gaussian_lattice-main/ROM_GPV$ ./signature_tests 1
===== Parameters =====
n = 1024
q = 1073707009
k = 30
d = 1
sigma = 7.000000
zeta = 83832.000000
alpha = 48.340000
=====

Generating keys (A, T, cplx_T, sch_comp)...
Printing H...
[[#(0, 0) :
[133558822, 130765448, 221321630, 455321427, 271032059, 880093465, 290643902, 1019902526, 470829834, 428656885, 184794443, 345821570
, 563509366, 1036365351, 863843855, 106511042, 1004892702, 290323438, 877870179, 398436949, 907933143, 115544059, 67535380, 74691105
1, 56163997, 446579699, 958837374, 985275699, 727547935, 534533886, 470360606, 981515021, 713583664, 540868587, 897361516, 338315960
, 487658679, 630463757, 550021265, 315499966, 691882579, 378238909, 146804610, 973201843, 339602605, 86880468, 375461995, 816720582,
82247405, 4638224105, 850723377, 465407863, 451409485, 34582255, 529606411, 656744340, 947211671, 214686517, 193208603, 969188949, 3
9070210, 758722268, 685251418, 770480696, 813703614, 511009381, 558348891, 746226684, 459044975, 585223166, 486015694, 134004234, 73
1371645, 306169273, 1066821062, 464741095, 824895704, 730498036, 4090337, 495988306, 1056832527, 544482745, 482591320, 1066636046, 1
026086889, 156279936, 18857331, 972197588, 591160723, 500222455, 599029638, 883537161, 13012740, 563284743, 221061231, 873128267, 44
2444427, 358158380, 224182129, 673178574, 662714945, 982179236, 596138217, 630964214, 233678365, 303239624, 930246951, 535543955, 59
3407357, 598822299, 1066116776, 475799905, 791401665, 305544599, 772909677, 427818089, 693920941, 1015826067, 541757105, 950832698,
399713293, 892079203, 1071561969, 526125805, 610171728, 771526084, 1000633639, 523348600, 601302808, 1044493237, 194098363, 10287896
86, 80425481, 839653646, 948609337, 945588840, 878562954, 1028311928, 973874080, 858268369, 170852368, 986092460, 473618292, 6170550
87, 569009013, 708063077, 657687198, 304571486, 223846575, 161176934, 1040327603, 423799095, 241364420, 715822450, 814637589, 123959
202, 88358310, 47076264, 1019477284, 452499893, 620957527, 841589801, 878549003, 1060126685, 634143621, 973069537, 932150023, 298701
877, 463265865, 831640518, 213823401, 673494864, 297440142, 1019042405, 62388566, 434526734, 109532770, 1018761963, 146693140, 54543
9204, 624427067, 413611365, 1005674295, 236188519, 814856868, 293144629, 143487873, 393171553, 523472600, 952634614, 491066497, 4338
27147, 35356574, 747006067, 279819084, 305728779, 1052276932, 459492052, 955749987, 801631410, 506617419, 486990640, 676343533, 8528
90588, 227083288, 796369128, 351110031, 116246139, 409968834, 1014148271, 695842362, 44301320, 660451360, 1001621708, 707674709, 199
130112, 293342345, 153644803, 461465682, 511542833, 928977417, 516669543, 261733222, 155637909, 185521002, 20903127, 795908226, 1038
632867, 426417141, 126510118, 361473174, 693908038, 690595236, 491612571, 170000954, 686812330, 62975623, 165723882, 488651848, 8792
604, 699584681, 430444103, 882323300, 794179007, 1060095254, 644179604, 67560811, 226375076, 586620548, 904372672, 620157385, 433661
784, 446784223, 80878874, 235871449, 488020478, 373727698, 636698907, 925403215, 291867220, 612037317, 449118859, 677461807, 4407075
7, 36729298, 637185495, 207309644, 670970981, 886728283, 473719817, 397791968, 657803076, 1001347056, 285176630, 380212433, 53126152
1, 374325988, 60936018, 143601125, 629024476, 2685127, 607237278, 220453224, 329572357, 1045001404, 766037179, 1046989653, 870843666
, 516241156, 339363419, 98799693, 515305784, 1051461195, 953226254, 816289530, 523618806, 929592919, 496560723, 952527861, 600851976
, 312388967, 18291891, 620981673, 837463328, 988301785, 187896292, 581133766, 649429930, 532190218, 969965147, 925754881, 571466088,
769623953, 228339484, 965225906, 775343922, 870780520, 181087679, 488170329, 188095133, 792663096, 834256134, 746798420, 144310743,
631901335, 418018021, 707866042, 577236856, 619895605, 466029003, 330838849, 1067075196, 7923466, 809283156, 285655254, 655173372,
441715717, 948693454, 233274209, 82225452, 143494239, 23884066, 311360306, 311142369, 656771154, 317526490, 616423118, 1039469043, 3
85341558, 806351598, 248021611, 856254394, 31151480, 859405498, 809150409, 954775098, 854042081, 803196314, 832824270, 291286789, 65
053470, 1026216748, 343840045, 351621194, 994127844, 319377676, 384059465, 842983605, 19815368, 545417474, 636841286, 316927670, 407
644635, 1011821234, 216013459, 700050187, 568735509, 321419416, 839793558, 124944316, 543573380, 491628406, 335155742, 362695995, 55
0362216, 220810341, 51739190, 689847320, 329722749, 75792742, 431560679, 828128712, 204172292, 405352875, 324084524, 109888650, 1396
96390, 180504090, 1015540006, 478305511, 552848633, 697949245, 314235998, 370030022, 452019551, 591347561, 616918971, 1009421916, 89
0862892, 762899362, 249949165, 93674672, 830618974, 467707324, 314428720, 1010301318, 138548496, 950309262, 204320392, 812612212, 23

```

- ▷ The modified `Sign2()` produces a signature on some message. The signature is of the form $\sigma = (\sigma_1, \sigma_2, \sigma_3)$.

```

deepesh@deepesh:~/Downloads/module_gaussian_lattice-main/ROM_GPV$ ./signature_tests 1
===== Parameters =====
n = 1024
q = 1073707009
k = 30
d = 1
sigma = 7.000000
zeta = 83832.000000
alpha = 48.340000
=====

Generating keys (A, T, cplx_T, sch_comp)...
Done
Generating a message m...
Done
Signing...
Printing Sig_1...
[[#(0, 0) :
[833150016, 455330446, 57164192, 461546075, 309576612, 698354151, 375065144, 166755110, 1052707177, 242364828, 906016896, 725744791,
231537953, 245870807, 747650981, 387166703, 751811304, 72491092, 724692367, 11142589, 250597150, 432297937, 982419111, 92022359, 64
6761125, 758584826, 613457599, 225498449, 123824293, 739560125, 540031597, 885465512, 674585894, 497582273, 392582873, 948139662, 19
8924975, 296074851, 263213836, 55816130, 47602229, 598205957, 1016001521, 384928202, 615262967, 609201005, 448412894, 23430411, 5857
21297, 589689188, 944610562, 629582979, 625493051, 4808083, 672114901, 149552345, 658081487, 784291734, 537141176, 524605148, 765987
034, 781153147, 342401524, 634223066, 85177911, 441262141, 52326557, 23943963, 157414359, 575930892, 274071084, 124601769, 277790433
, 865948036, 211480364, 438912785, 641306081, 794373864, 961180201, 617052054, 793435033, 105741836, 52649307, 597827013, 33378789,
902640991, 750441872, 690007086, 945273503, 909899756, 55432648, 205477259, 818703057, 836879528, 691830071, 1047709863, 177928022,
151585315, 172505477, 711995436, 961670012, 40054846, 556560824, 411392864, 464201397, 816273411, 558760760, 975675120, 653269885, 7
62794222, 168626744, 1052347025, 408681054, 974782360, 882190723, 775129870, 1010568812, 932897432, 790408374, 216530893, 633096435,
896241737, 149208582, 987222288, 457820382, 340625592, 618685709, 904219589, 871368884, 798129033, 685607622, 125161571, 357908236,
890327125, 132903128, 243652919, 760412863, 336652970, 902862419, 924209237, 182793739, 402540528, 846638309, 874446170, 3143017, 2
86057910, 10647267, 710118005, 883802919, 504890164, 333738467, 193042688, 444030498, 613666074, 562142520, 36977724, 428202515, 303
898860, 242457412, 857933944, 22675500, 958308797, 422873870, 589977706, 251031929, 288541273, 78034317, 689938065, 747070818, 25178
1355, 163346983, 567999866, 552057832, 711185475, 323732926, 308831252, 693877324, 428515007, 490938293, 538860259, 436452122, 18945
9100, 818358473, 85385847, 723746327, 901859133, 282124408, 1023177322, 662128389, 804825436, 397190505, 754197076, 818986470, 15747
6827, 849724194, 676974027, 544232172, 636362242, 863160667, 6743187, 191231218, 91484225, 29909887, 981726535, 878595597, 467528261
, 224074637, 1030761655, 224587913, 401052115, 140525537, 36716249, 54815445, 451445386, 194355853, 525942783, 13411484, 476324069,
501116590, 645603477, 969916636, 168380891, 46884221, 1062300282, 583971653, 627933639, 401040561, 350990190, 741021320, 823033910,
580282552, 1028742218, 338494486, 736953918, 463977213, 48756922, 320298551, 45276743, 966951562, 396159241, 410613349, 184342496, 8
17274904, 450806762, 383235053, 349723469, 779168255, 951767353, 134112649, 723056662, 706459406, 1035132962, 1006100174, 1049711871
, 597762545, 977105274, 584950373, 725230996, 462998749, 279806980, 762675056, 996234239, 483922410, 981751100, 569785042, 619284556
, 458427552, 536694527, 44559425, 2548694, 835492917, 467423507, 197886916, 679606245, 477729620, 34667665, 1021934773, 101544266, 4
43918016, 387832444, 445743726, 129606536, 826270982, 426203706, 579071118, 828372523, 379776685, 867059490, 738470818, 279183342, 1
76251707, 34679874, 914955365, 999106202, 16541282, 249181657, 887900173, 254718086, 977530513, 274371120, 922217229, 544533395, 106
9479678, 858164684, 640780345, 30461593, 487850442, 605159626, 457412034, 335825398, 418768811, 1002987065, 546047726, 625217657, 31
4094557, 118622332, 363862294, 879503909, 953296335, 484356629, 20935412, 258797018, 715680470, 8975752, 262708420, 703625839, 94763
2532, 15312738, 538930379, 797002738, 599317614, 449782837, 897581276, 204522051, 853754105, 170511494, 478149436, 93790067, 6460255
67, 254000812, 858077633, 601472640, 816338190, 947627832, 366516118, 788461376, 861861082, 128892956, 729906933, 729530568, 3500101
2, 92867438, 24101356, 321332218, 342029817, 454578714, 869241221, 250410306, 925507384, 894845183, 157043194, 820659640, 927047147,
372929616, 902416183, 1046447481, 608525501, 549756431, 109730933, 283976780, 433853008, 909654395, 94498000, 869422729, 740166815,

```

-
- ▷ The modified `Verify2()` outputs 1 on successful verification and outputs 0 otherwise.

```
deepesh@deepeshg:~/Downloads/ROM_GPV$ ./signature_tests 10
===== Parameters =====
n = 1024
q = 1073707009
k = 30
d = 1
sigma = 7.000000
zeta = 83832.000000
alpha = 48.340000
=====
Generating keys (A, T, cplx_T, sch_comp)...
Done
Generating a message m...
Done
Signing...
Ch=0 testing
verification successful @ 0
Ch=0 testing
verification successful @ 1
Ch=1 testing
verification successful @ 2
Ch=2 testing
verification successful @ 3
Ch=2 testing
verification successful @ 4
Ch=1 testing
verification successful @ 5
Ch=2 testing
verification successful @ 6
Ch=2 testing
verification successful @ 7
Ch=2 testing
verification successful @ 8
Ch=2 testing
verification successful @ 9
```

5 Conclusion

We have successfully modified the Key Generation, Sign, and Verify functions of the undeniable signature scheme for modular lattices.

References

- [1] P. Bert, G. Eberhart, L. Prabel, A. Roux-Langlois, and M. Sabt. Implementation of lattice trapdoors on modules and applications. In *PQCrypto 2021 - International Conference on Post-Quantum Cryptography*, pages 195–214, 2021. Virtual event, France.
- [2] Aditi Gangopadhyay. Development of lattice-based encryption and digital signature algorithms for quantum-safe cryptography: A drdo sponsored project. Report, 2021. Submitted by Dr. Aditi Gangopadhyay (Principal Investigator), Department of Mathematics, Indian Institute of Technology Roorkee, Roorkee - 247667, Uttarakhand, India.
- [3] C. Gentry, C. Peikert, and V. Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. *Stanford University, SRI International, MIT*, 2007.
- [4] V. Lyubashevsky. Lattice-based digital signatures. *Natl Sci Rev*, 8:nwab077, 2021. Special Topic: Lattice-based Cryptography and the Underlying Computational Problems.